

[Chap 7] 모델의 성능지표



1종 오류와 2종 오류

- 늑대와 양치기 소년
 - 처음에는 늑대가 없는데도 "늑대가 나타났어요!"라는 말에 마을 사람들이 달려감
 - 두 번째도 늑대가 없는데도 "늑대가 나타났어요!"라는 말에 마을 사람들이 달려감
 - 세 번째는 늑대가 나타났는데도 "늑대가 나타났어요!"라는 말을 마을 사람들이 믿지 않음
 - 마을 사람들은 세 번의 판단을 했고, 세 번 다 틀림
- 귀무가설과 대립가설
 - 대립가설(alternative hypothesis): 데이터를 보고, 기존과 특별한 차이(평균이 다르다거나)가 있다고 생각하는 가설
 - 귀무가설(null hypothesis): 기존과 특별한 차이가 없다고 생각하는 가설
 - 마을 사람들이 양치기 소년의 듣고, '늑대가 있다'라고 생각하는 것이 대립가설, '늑대가 없다'라고 생각하는 것이 귀무가설이 됨
- 마을 사람들의 오류
 - 첫 번째와 두 번째 경우: 늑대가 있다(positive)고 예측했는데 틀림(실제는 negative)
→ 1종 오류 (Type I error), 오경보(false alarm)
 - 세 번째 경우: 늑대가 없다(negative)라고 예측했는데 틀림(실제는 positive)
→ 2종 오류(Type II error), 탐지 누락(missed detection)
 - 통계학의 p-value는 1종 오류의 확률로 정의됨

혼동행렬

- OX퀴즈의 예: 10문제에 대한 채점표

〈표 7-1〉 OX퀴즈 채점표

구분	1번	2번	3번	4번	5번	6번	7번	8번	9번	10번
정답	X	X	X	X	O	O	O	X	X	O
답안지	X	X	X	O	O	O	X	X	O	O
정오구분	정답	정답	정답	오답	정답	정답	오답	정답	오답	정답

- 이 학생은 10문제 중에 7문제를 맞추고 3문제를 틀려서 70점
- 틀린 유형에 2가지(정답이 O인데 X로 쓴 경우와 정답이 X인데 O로 쓴 경우), 맞는 유형에 2가지(정답이 O인데 O로 쓴 경우와 정답이 X인데 X로 쓴 경우)가 있음
- 정오답 유형별 채점표

〈표 7-2〉 정오답 유형별 채점표

정답 \ 답안지	X	O
X	4	2
O	1	3

3

혼동행렬 (계속)

- 정답을 실제값(actual value), 학생의 답안을 예측값(predicted)이라고 하고, O(True)를 숫자 1, X(False)를 숫자 0으로 나타내면 다음과 같음

〈표 7-3〉 OX퀴즈에 대한 혼동행렬

Actual \ Predicted	0	1
0	4	2
1	1	3

- 이것을 혼동행렬(confusion matrix)이라고 함
- 혼동행렬 계산 코드

```
1 from sklearn.metrics import confusion_matrix
2 y_true = [0,0,0,0,1,1,1,0,0,1]
3 y_pred = [0,0,0,1,1,1,0,0,1,1]
4 print(confusion_matrix(y_true, y_pred))
```

```
[[4 2]
 [1 3]]
```

4

TP, TN, FP, FN

- 맞는 경우 2가지, 틀리는 경우 2가지를 각각 구분하여 TP, TN, FP, FN으로 일컬음
 - 앞 글자의 T 또는 F는 True 및 False의 약자로서, 실제로 맞았는지 틀렸는지를 나타냄
 - 뒤 글자의 P와 N은 각각 Positive, Negative의 약자로서 나의 답안지, 즉 내가 예측한 값을 말함

- 아래 혼동행렬에서의 TP, TN, FP, FN

〈표 7-4〉 혼동행렬의 예시

Actual \ Predicted	0 (Negative)	1 (Positive)
0 (Negative)	4 (TN)	2 (FP)
1 (Positive)	1 (FN)	3 (TP)

- TP(True Positive): Positive라고 답했는데 맞은 경우의 횟수 (TP=3)
- TN(True Negative): Negative라고 답했는데 맞은 경우의 횟수 (TN=4)
- FP(False Positive): Positive라고 답했는데 틀린 경우의 횟수 (FP=2)
- FN(False Negative): Negative라고 답하고 틀린 경우의 횟수(FN=1)

5

정확도(accuracy)

- 예측한 총 개수 대비 맞은 개수의 비율
 - OX 시험 점수와 같은 의미
- 정확도 = $(TP + TN) / (TP + TN + FP + FN)$
 - 혼동행렬 대각선의 값을 전체 합계로 나눈 것
 - 아래 그림에서는 $(3 + 4)/10 = 70\%$

Actual \ Predicted	0 (Negative)	1 (Positive)
0 (Negative)	4 (TN)	2 (FP)
1 (Positive)	1 (FN)	3 (TP)

6

정확도의 문제점

- 정확도가 높다고 꼭 좋은 모델이라고 보기 어려움
- 예) 외계인을 판별하는 특수안경 모델 A의 정확도가 99.1%라면 좋은 성능?
 - 이 특수안경의 혼동행렬

〈표 7-5〉 외계인 판별 안경 A의 혼동행렬

Actual \ Predicted	0 (지구인)	1 (외계인)
0 (지구인)	990	0
1 (외계인)	9	1

- 외계인 10명 중에 1명만 제대로 판별
 - 상식적으로 성능이 좋지 않음에도 정확도는 99.1%로 나타남
- Positive가 전체 negative 중에서 매우 희소한 경우에 이런 문제가 발생
 - 중요한 판단은 희소한 대상을 판별해야 하는 경우가 많으므로(예: 바이러스 검사, 6-sigma 수준의 불량률), 정확도만으로 이야기하는 것은 위험

7

정밀도(precision)와 재현율(recall)

- 정밀도(precision)
 - 모델이 positive라고 예측했을 때 이 중에서 정답인 비율
 - 정밀도 = $TP / (TP + FP)$
 - 정밀도가 높다는 것은 1종 오류가 낮음, 즉 오경보(false alarm)가 적다는 의미
 - 앞의 외계인 탐지안경 예제에서 정밀도 = 100% (1건 검출한 것이 정답이었음)
- 재현율(recall)
 - 모델이 실제로 positive인 값들 중에서 몇 %를 검출했는지를 측정
 - 재현율 = $TP / (TP + FN)$
 - 재현율이 높은 것은 2종 오류가 낮음, 즉 탐지 누락(missed detection)이 적다는 의미
 - 앞의 외계인 탐지안경 예제에서 정확도 = $1 / (1 + 9) = 10\%$ 에 불과

8

종합적 성능 판단의 필요성

- 예) 외계인 탐지 안경 모델 B의 성능 측정

〈표 7-6〉 외계인 판별 안경 B의 혼동행렬

Actual \ Predicted	0 (지구인)	1 (외계인)
0 (지구인)	981	9
1 (외계인)	1	9

- 정확도 = $(981 + 9) / 1000 = 99.0\%$
- 정밀도 = $9 / (9 + 9) = 50\%$
- 재현율 = $9 / (1 + 9) = 90\%$

지표	모델 A	모델 B
정확도(accuracy)	99.1%	99.0%
정밀도(precision)	100%	50%
재현율(recall)	10%	90%

- 모델의 성능 평가는 단순한 문제가 아님
 - 모델의 용도를 비추어 생각해보면, 이 경우는 모델 B가 더 좋은 모델임
 - 확실하지 않은 경우에도 일단 외계인이라고 식별해 재현율을 높이는 것이 바람직
 - 경우에 따라 정밀도, 재현율 등을 종합적으로 고려하여 성능을 판단해야 함

9

F1 점수(F1 score)

- Positive와 negative 사이의 불균형이 심할 때 정확도는 의미없는 지표가 됨
 - 데이터의 불균형이 있을 때에도 종합적인 값을 주는 지표의 필요성

- F1 점수(F1 score)
 - 정밀도와 재현율의 조화평균(harmonic mean)

$$F1 \text{ 점수} = 2 \times \text{재현율} \times \text{정밀도} / (\text{재현율} + \text{정밀도})$$

- 조화평균은 한 쪽이 극단적인 값이면 낮아지는 특성이 있음
 - 이에 따라, 단순 평균보다 의미있는 값을 제공
 - 예로서 재현율 100%, 정밀도 0%의 경우라면 F1 점수는 0이 됨
- 외계인 탐지 안경 A와 B의 비교
 - 모델 A의 F1 점수 = $2 \times 1 \times 0.1 / (1 + 0.1) = 0.182$
 - 모델 B의 F1 점수 = $2 \times 0.5 \times 0.9 / (0.5 + 0.9) = 0.643$
 - 모델 B의 F1 점수가 더 높게 나타남

성능지표 계산 코드

- OX퀴즈에 대한 성능지표를 출력

예제 ex7-1.py

```
1 from sklearn.metrics import confusion_matrix
2 from sklearn.metrics import classification_report
3 y_true = [0,0,0,0,1,1,1,0,0,1]
4 y_pred = [0,0,0,1,1,1,0,0,1,1]
5 print(confusion_matrix(y_true, y_pred))
6 print(classification_report(y_true, y_pred))
```

```
[[4 2]
 [1 3]]
```

		precision	recall	f1-score
	0	0.80	0.67	
	1	0.60	0.75	
accuracy				
macro avg		0.70	0.71	
weighted avg		0.72	0.70	

- sklearn의 confusion_matrix와 classification_report 기능 활용
- 1을 양성으로 보았을 때 정밀도 60%, 재현율 75%, F1 점수 67%로 나타남
- support는 해당 값이 몇 건 있었는지를 나타냄: 1이 4건 있었음을 표현
- 전체 정확도는 70%로 계산

11

성능지표 계산 코드

예제 ex7-1.py

```
1 from sklearn.metrics import confusion_matrix
2 from sklearn.metrics import classification_report
3 y_true = [0,0,0,0,1,1,1,0,0,1]
4 y_pred = [0,0,0,1,1,1,0,0,1,1]
5 print(confusion_matrix(y_true, y_pred))
6 print(classification_report(y_true, y_pred))
```

```
[[4 2]
 [1 3]]
```

		precision	recall	f1-score	support
	0	0.80	0.67	0.73	6
	1	0.60	0.75	0.67	4
accuracy					
macro avg		0.70	0.71	0.70	10
weighted avg		0.72	0.70	0.70	10

12

ROC 곡선과 AUC

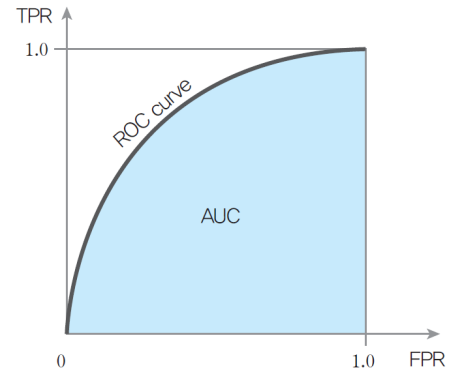
- 참 양성 비율(TPR)과 허위 양성 비율(FPR)
 - 참 양성 비율(True Positive Rate): 재현율의 동의어

$$TPR = \frac{TP}{TP + FN}$$

- 허위 양성 비율(False Positive Rate): negative 중에서 positive로 잘못 판별한 비율

$$FPR = \frac{FP}{FP + TN}$$

- ROC 곡선 (ROC: Receiver Operating Characteristic)
 - 분류 임계값에 따른 분류 모델의 성능 시각화
 - 모델이 positive와 negative를 구분하는 임계값 (threshold)에 따른 TPR과 FPR을 나타낸 그래프



[그림 7-3] ROC 곡선의 예

13

ROC 예제

- 대사증후군 내원환자의 키와 몸무게
 - BMI(= 몸무게 / 키²)가 30 이상이면 고도비만
 - 몸무게만으로 고도비만여부를 판정해야 한다면?

〈표 7-8〉 ROC 예제 데이터

환자번호	몸무게(kg)	키(cm)	BMI	진단결과
1	58	160	22.7	
2	62	163	23.3	
3	69	151	30.3	고도비만
4	73	175	23.8	
5	77	178	24.3	
6	84	164	31.2	고도비만
7	88	165	32.3	고도비만
8	92	193	24.7	
9	96	172	32.4	고도비만
10	104	183	31.1	고도비만

ROC 예제 (계속)

- 고도비만으로 판정하는 몸무게 기준을 80kg로 하였을 때

〈표 7-9〉 임계값 80kg의 혼동행렬

Actual \ Predicted	0 (정상)	1 (고도비만)
0 (정상)	4 (TN)	1 (FP)
1 (고도비만)	1 (FN)	4 (TP)

– $TPR = 4/(1+4) = 0.8$, $FPR = 1/(1+4) = 0.2$

- 임계값을 55kg으로 하면? → $TPR = FPR = 1$
- 임계값을 105kg으로 하면? → $TPR = FPR = 0$
- 임계값을 다양하게 바꾸면서 TPR, FPR을 계산하여 점을 연결하면 ROC 곡선이 됨

15

ROC 그래프 코드

- TPR, FPR 계산
 - sklearn.metrics에서 혼동행렬 모듈 confusion_matrix를 가져옴
- 환자의 몸무게를 weight, 키를 height라는 이름의 리스트에 넣음
- 각 환자의 고도비만 여부를 나타내는 정답값을 y_true에 넣음(0: 정상, 1:고도비만)
- 고도비만 판단 몸무게의 임계값을 80으로 정하여 threshold라는 변수에 넣음

```
from sklearn.metrics import confusion_matrix
```

```
1 weight = [58,62,69,73,77,84,88,92,96,104]
2 height = [160,163,151,175,178,164,165,193,172,183]
```

```
y_true = [0,0,1,0,0,1,1,0,1,1]
```

```
threshold=80
```

16

ROC 그래프 코드 (계속)

- TPR, FPR 계산 (계속)
 - 각 환자에 대해 몸무게가 threshold 이상이면 1을, 아니면 0을 y_pred에 넣음

```
1 y_pred=[]
2 for i in range(10):
3     w = weight[i]
4     if w>=threshold: y = 1
5     else: y = 0
6     y_pred.append(y)
```

- 이렇게 예측한 y_pred와 정답값 y_true를 비교하여 혼동행렬을 출력

```
1 cm = confusion_matrix(y_true, y_pred)
2 print(cm)
```

17

ROC 그래프 코드 (계속)

- TPR, FPR 계산 (계속)
 - 이 혼동행렬에서 TN, TP, FN, FP를 읽어옴

```
1 tn = cm[0][0]
2 tp = cm[1][1]
3 fn = cm[1][0]
4 fp = cm[0][1]
```

- TPR과 FPR 값을 계산하여 출력

```
1 tpr = tp / (tp+fn)
2 fpr = fp / (fp+tn)
3 print(tpr, fpr)
```

```
[[4 1]
 [1 4]]
0.8 0.2
```

18

ROC 그래프 코드 (계속)

- ROC 곡선 출력 프로그램
 - 모듈을 들여오고 데이터를 정의하는 부분은 동일

```
1 from sklearn.metrics import confusion_matrix
2
3 weight = [58,62,69,73,77,84,88,92,96,104]
4 height = [160,163,151,175,178,164,165,193,172,183]
5 y_true = [0,0,1,0,0,1,1,0,1,1]
```

- 임계값을 변화시키는데 따른 FPR과 TPR을 저장할 빈 리스트를 초기화

```
1 fpr_list=[]
2 tpr_list=[]
```

19

ROC 그래프 코드 (계속)

- ROC 곡선 출력 프로그램 (계속)
 - threshold의 값을 55에서 105까지 5씩 변화시키면서 y_pred를 계산하고 이에 대한 FPR, TPR을 계산해서 차례로 fpr_list, tpr_list에 저장

- (2~14행) 주어진 threshold 값에 대해 tpr과 fpr 값을 계산
- (15행) threshold와 tpr, fpr 값을 출력
- (17~18행) 계산된 fpr과 tpr을 차례로 fpr_list, tpr_list에 기입

```
1 for threshold in range(55,110,5):
2     y_pred=[]
3     for i in range(10):
4         w = weight[i]
5         if w>=threshold: y = 1
6         else: y=0
7         y_pred.append(y)
8     cm = confusion_matrix(y_true, y_pred)
9     tn = cm[0][0]
10    tp = cm[1][1]
11    fn = cm[1][0]
12    fp = cm[0][1]
13    tpr = tp / (tp+fn)
14    fpr = fp / (fp+tn)
15    print(threshold, fpr, tpr)
16
17    fpr_list.append(fpr)
18    tpr_list.append(tpr)
```

20

ROC 그래프 코드 (계속)

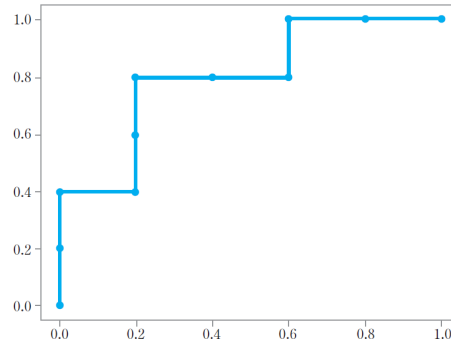
- ROC 곡선 출력 프로그램 (계속)
 - 계산한 fpr_list를 가로축에, tpr_list를 세로축에 놓고 점을 찍어서 이으면 ROC 곡선이 그려짐

```
1 import matplotlib.pyplot as plt
2 plt.plot(fpr_list, tpr_list, '-o')
3 plt.show()
```

- (2행) -o 옵션으로 선을 이으면서 각각의 점을 표시하도록 함

출력 결과

```
55 1.0 1.0
60 0.8 1.0
65 0.6 1.0
70 0.6 0.8
75 0.4 0.8
80 0.2 0.8
85 0.2 0.6
90 0.2 0.4
95 0.0 0.4
100 0.0 0.2
105 0.0 0.0
```

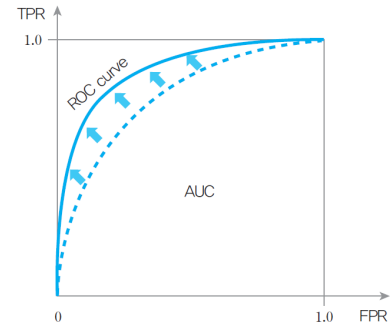


[그림 7-4] 예제프로그램 7-3의 ROC 곡선 출력

21

ROC 그래프 코드 (계속)

- 모델 성능과 ROC
 - 예측 모델이 정확할수록 TPR은 높고 FPR은 낮으므로, 좋은 모델일수록 (FPR, TPR) 점들이 왼쪽 위로 이동하여 ROC 곡선 아래의 면적 AUC는 넓어지게 됨



[그림 7-5] 모델 개선과 ROC와 AUC의 변화

- 개선된 모델: BMI 지수를 계산하여 고도비만 판단

예제 ex7-4.py

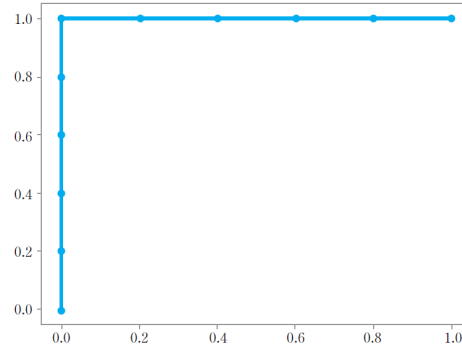
```
...
10 for threshold in np.arange(20, 40, 0.1):
11     y_pred=[]
12     for i in range(10):
13         bmi = weight[i] / (height[i]/100)**2
14         if bmi>=threshold: y = 1
15         else: y=0
...
```

range는 정수로만
증가시킬 수 있어,
np.arange 사용

몸무게 대신
BMI 공식으로
1,0 판정

ROC 그래프 코드 (계속)

- 개선된 모델 (계속)
 - 모든 점들이 더이상 이동할 수 없을때까지 왼쪽 위로 이동해서 붙어있음
 - 고도비만 여부의 진단 기준이 원래 BMI 지수이므로, BMI를 계산해서 예측하면 정답과 완전히 동일



[그림 7-6] 개선된 모델의 ROC 곡선

- 가장 좋은 threshold 값은?
 - BMI값 threshold 24.7부터 30.2까지에서 FPR은 0, TPR은 1로 나타나, 오류 없이 완벽한 판단을 내림
 - BMI 기준으로 비만 여부를 판단하는 threshold 값은 24.9에서 30.2 사이의 값이 되어야 함을 의미
 - 실제로 BMI의 고도비만 판정기준은 약 30.0